Squid proxy

Technical Report: Squid Forward Proxy Home Lab

Project: Implementation of a Squid Forward Proxy
Virtualization Platform: Oracle VirtualBox
Proxy Server: Ubuntu Server
Client Machine: Kali Linux
Proxy Port: 3128
Lab Subnet: 192.168.10.0/24

---

1. Introduction

The purpose of this project was to implement a forward proxy using Squid in a virtualized home lab. Ubuntu Server was used as the proxy server, while Kali Linux was used as the client.

The project focused on understanding how a forward proxy works, configuring network connectivity between virtual machines, controlling access to websites, requiring users to authenticate, and testing proxy behavior.

2. Lab Architecture

The lab was built using Oracle VirtualBox with two virtual machines.

Machine	Operating system	Role
Machine 1	Ubuntu Server	Squid forward proxy
Machine 2	Kali Linux	Client

Ubuntu Server was configured to provide the proxy service. Kali Linux was used to send web requests through the proxy.

The intended network design used a NAT adapter for internet connectivity on Ubuntu Server and a separate lab network for communication between Ubuntu Server and Kali Linux.

The proxy address used for client requests was `192.168.10.1`, with TCP port `3128`.

Network traffic flow

Kali Linux → Ubuntu Server (Squid) → Internet → Destination website

When Kali Linux sends a request through Squid, the proxy evaluates its access-control rules before forwarding the request to the destination website. With authentication enabled, the proxy also checks the user's credentials during this evaluation.

3. Squid Installation and Configuration

Squid was configured on Ubuntu Server. Its primary configuration file is:

`/etc/squid/squid.conf`

Before modifying the configuration, a backup was created using:

```
sudo cp /etc/squid/squid.conf /etc/squid/squid.conf.bak
```

The configuration file was opened for editing with:

```
sudo nano /etc/squid/squid.conf
```

3.1 Basic proxy configuration

The following directives were used to define the proxy port and client network:

```
http_port 3128

acl localnet src 192.168.10.0/24

http_access allow localhost
http_access allow localnet
http_access deny all
```

The directives have the following purposes:

- `http_port 3128` specifies the TCP port on which Squid listens for proxy requests.
- `acl localnet src 192.168.10.0/24` defines the client subnet permitted to use the proxy.
- `http_access allow localhost` permits matching requests originating from the local machine.
- `http_access allow localnet` permits requests from the lab subnet.
- `http_access deny all` denies requests that have not been permitted by previous rules.

The order of the access rules determines how Squid evaluates incoming requests. Squid checks the rules from top to bottom and stops at the first rule that matches.

4. Website Blocking

After configuring the basic proxy, domain-based access control was used to restrict access to selected websites.

The following configuration blocks Facebook and YouTube for requests passing through Squid:

```
http_port 3128

acl localnet src 192.168.10.0/24

acl blocked_sites dstdomain .facebook.com .youtube.com

http_access deny blocked_sites
http_access allow localhost
http_access allow localnet
http_access deny all
```

The `blocked_sites` ACL identifies the domains to restrict. The `http_access deny blocked_sites` directive rejects matching requests before the general allow rules are evaluated.

This configuration demonstrates how a forward proxy can enforce a website-blocking policy for clients using the proxy.

5. Allowing One Website and Blocking Others

A second access-control policy was created to allow only Kali Linux's official website through Squid.

The configuration was:

```
http_port 3128

acl localnet src 192.168.10.0/24

acl allowed_site dstdomain .kali.org

http_access allow localhost allowed_site
http_access allow localnet allowed_site
http_access deny all
```

The `allowed_site` ACL matches `kali.org` and its subdomains.

The configuration allows requests to the specified domain from localhost and the lab subnet. The final `http_access deny all` directive denies all other requests handled by Squid.

This approach is known as an **allowlist policy** because access is granted only to explicitly permitted destinations.

6. Proxy Authentication

So far, any machine in `192.168.10.0/24` could use the proxy without proving who it was. Authentication was added so that each user must also provide a valid username and password.

6.1 Purpose

Authentication gives the proxy a way to identify users, not just IP addresses. This allows:

- Restricting the proxy to known users only.
- Recording which user made each request in the access log.
- Building per-user rules later (for example, different access for different people).

6.2 Installing the password tool

The `htpasswd` tool is part of the `apache2-utils` package:

```
sudo apt install apache2-utils
```

6.3 Creating the password file

The first user was created with the `-c` option, which creates the file:

```
sudo htpasswd -c /etc/squid/passwords alice
```

Additional users were added **without** `-c`, because `-c` would overwrite the existing file:

```
sudo htpasswd /etc/squid/passwords bob
```

Squid's authentication helper runs as the `proxy` user, so the file needs the correct owner and permissions:

```
sudo chown proxy:proxy /etc/squid/passwords
sudo chmod 640 /etc/squid/passwords
```

Passwords are stored as hashes, not as plain text.

6.4 Testing the helper manually

Before changing Squid, the helper program was tested on its own:

```
/usr/lib/squid/basic_ncsa_auth /etc/squid/passwords
```

Typing `alice password` (username and password separated by a space) should return `OK`. A wrong password returns `ERR`. This confirms the password file works before Squid is involved.

6.5 Configuration

The following configuration combines authentication, the network restriction, and the blocklist:

```
http_port 3128

acl localnet src 192.168.10.0/24
acl blocked_sites dstdomain .facebook.com .youtube.com

auth_param basic program /usr/lib/squid/basic_ncsa_auth /etc/squid/passwords
auth_param basic realm Squid Proxy Authentication
auth_param basic credentialsttl 2 hours

acl authenticated proxy_auth REQUIRED

http_access deny blocked_sites
http_access deny !authenticated
http_access allow localhost
http_access allow localnet
http_access deny all
```

6.6 Explanation of the directives

Directive	Purpose
`auth_param basic program ...`	Selects Basic authentication and names the helper program that checks credentials against the password file.
`auth_param basic realm ...`	Text shown in the login prompt of the client.
`auth_param basic credentialsttl 2 hours`	Squid caches a successful credential check for 2 hours, so it does not call the helper for every request.
`acl authenticated proxy_auth REQUIRED`	Defines an ACL that matches any request carrying a **valid** username and password. `REQUIRED` means any valid user.
`http_access deny !authenticated`	Denies requests that are **not** authenticated. The `!` means NOT. This rule triggers the `407 Proxy Authentication Required` response.

The name `authenticated` is only a label chosen by the administrator. The ACL is not a list of users: it is a test that Squid runs on every request. If the request has valid credentials, the test matches. If not, it does not match.

Specific users can be matched instead of all valid users:

```
acl admins proxy_auth alice bob
```

6.7 How the rules are evaluated

Squid reads the `http_access` rules from top to bottom and stops at the first match:

1. `deny blocked_sites` : blocked domains are denied for everyone, with a plain `403 Forbidden` .
2. `deny !authenticated` : users without valid credentials receive `407` , and the client asks for a login.
3. `allow localhost` : the proxy machine itself is allowed.
4. `allow localnet` : clients in `192.168.10.0/24` are allowed.
5. `deny all` : everything else is denied.

Matching `authenticated` does **not** mean the user is allowed. It only proves the login was correct. Access is granted only by an `allow` rule. A user with a valid password who is outside `192.168.10.0/24` is still denied by `deny all` .

Access therefore requires **both** valid credentials **and** a client address inside the allowed network.

6.8 Limitations

- **Basic authentication is not encrypted.** Credentials are only Base64-encoded, so anyone capturing traffic on the network can read them. This is acceptable in an isolated lab but not on an untrusted network. Section 7 explains this in detail and shows how to protect the login with an SSH tunnel.
- **Credential caching.** Because of `credentialsttl 2 hours` , a deleted user or a changed password may keep working for up to 2 hours, until Squid is restarted or reconfigured.
- **Rule order matters.** Because `deny blocked_sites` comes first, blocked sites return `403` even to users who have not logged in.

7. Securing the Proxy Connection with an SSH Tunnel

Section 6.8 noted that Basic authentication is not encrypted. This section explains the problem in detail and shows how an SSH tunnel was used to protect the proxy login.

7.1 The problem

When a client logs in to the proxy, it sends a header like this:

```
Proxy-Authorization: Basic YWxpY2U6cGFzczEyMw==
```

The text after `Basic` is only **Base64 encoding**, not encryption. It can be reversed instantly:

```
echo "YWxpY2U6cGFzczEyMw==" | base64 -d
```

Result:

```
alice:pass123
```

Property	Base64 (encoding)	Encryption
Purpose	Change the format	Hide the data
Needs a key	No	Yes
Can anyone reverse it	Yes, instantly	No, only with the key

The link between Kali and Squid (`192.168.10.x` to `192.168.10.1:3128`) is plain HTTP, so anyone capturing traffic on that network (for example with Wireshark or `tcpdump`) can read and decode the password.

HTTPS websites are only partly protected. For `https://www.kali.org` , the client first sends `CONNECT www.kali.org:443` to Squid in **plain text**, and this message carries the proxy password. The encrypted session starts only **after** the tunnel is established, so the proxy login is exposed before the encryption begins.

7.2 The solution: SSH local port forwarding

An SSH tunnel makes Kali talk to its **own** port 3128. SSH carries that traffic, encrypted, to Squid on Ubuntu:

```
Kali (127.0.0.1:3128) == encrypted SSH ==> Ubuntu (Squid on 127.0.0.1:3128)
--> Internet
```

7.3 Configuration on Ubuntu (proxy server)

Install the SSH server if it is not already present:

```
sudo apt install openssh-server
```

Make Squid listen **only on localhost**, so it is no longer reachable directly from the network. In `squid.conf`:

```
http_port 127.0.0.1:3128
```

Validate and apply:

```
sudo squid -k parse
sudo systemctl restart squid
```

A restart is used because a plain reload may not apply a change to the listening address.

7.4 Creating the tunnel on Kali (client)

```
ssh -N -L 3128:127.0.0.1:3128 youruser@192.168.10.1
```

The terminal looks frozen after login. This is normal: the tunnel works as long as the command keeps running.

Part	Meaning
`ssh`	Program that creates an encrypted connection to another machine.
`-N`	Do not run a remote command or open a shell. Only keep the tunnel open.
`-L`	Local port forwarding: open a port on the local machine and send its traffic through the tunnel.
`3128:127.0.0.1:3128`	`local_port : destination_host : destination_port`.
`youruser@192.168.10.1`	The Ubuntu server and the account used to log in.

The three values in `3128:127.0.0.1:3128` mean:

Value	Meaning
First `3128`	Port opened by SSH **on Kali**. The client connects here.
`127.0.0.1`	Where traffic goes **after it arrives at Ubuntu**.
Second `3128`	Port on that destination, where Squid listens.

The middle `127.0.0.1` is **not** Kali's localhost. The destination is resolved on the Ubuntu side of the tunnel, so it means Ubuntu's own localhost, where Squid is listening.

7.5 Path of one request

```
Client on Kali
   |  (1) connects to 127.0.0.1:3128 on Kali
   v
SSH client on Kali (listening on port 3128)
   |  (2) wraps the data in an encrypted SSH connection
   v
===== encrypted tunnel over the lab network =====
   |
   v
SSH server on Ubuntu
   |  (3) decrypts, then connects to 127.0.0.1:3128 on Ubuntu
   v
Squid on Ubuntu
   |  (4) checks login and rules, then fetches the website
   v
Internet
```

The client believes it is using a normal proxy on its own machine and does not know about the tunnel.

7.6 Testing the tunnel with curl

In a second terminal on Kali:

```
curl -I -x http://127.0.0.1:3128 -U alice:correctpass https://www.kali.org/
```

The request succeeds through the tunnel. Requests to blocked sites such as YouTube still return `403 Forbidden`, and requests without credentials still return `407`.

7.7 Using the tunnel in a web browser (Firefox)

1. Start the tunnel and leave the terminal open.
2. Open **Settings**, then **General**, then **Network Settings**, then **Settings...**
3. Select **Manual proxy configuration**.
4. Set **HTTP Proxy** to `127.0.0.1` and **Port** to `3128`.
5. Tick **Also use this proxy for HTTPS** and click **OK**.
6. Open a website. Because of `deny !authenticated`, Firefox shows a login popup (the `407` response). Enter the Squid username and password.

Checks:

- `https://www.youtube.com` returns the Squid `403` access-denied page.
- `sudo tail -f /var/log/squid/access.log` on Ubuntu shows the requests with the username.
- If the SSH tunnel is stopped, the browser reports that the proxy server is refusing connections, which confirms that all traffic goes through the tunnel.

7.8 Verifying the protection

A packet capture on Ubuntu's lab-network interface (replace `enp0s8` with the real interface name) shows the difference:

```
sudo tcpdump -i enp0s8 -A port 3128
```

- **Without the tunnel:** the `Proxy-Authorization: Basic ...` header is visible in plain text.
- **With the tunnel:** nothing appears on port 3128 (Squid now listens only on localhost). The traffic is on port 22 and is unreadable:

```
sudo tcpdump -i enp0s8 -A port 22
```

7.9 What is protected and what is not

- **Protected:** the link between Kali and Ubuntu, including the proxy password.
- **Not changed:** the link between Ubuntu and the internet.

Squid now sees every request as coming from `127.0.0.1` (the SSH server on Ubuntu), not from Kali's IP. The `localnet` rule therefore no longer identifies the client, and the `allow localhost` rule matches instead. The password check (`deny !authenticated`) still comes first, so authentication still protects the proxy. Also, per-client IP rules and the real client IP in the logs are lost, because every user appears as `127.0.0.1` . The username field in the log still identifies the user.

By default, SSH opens the local port only on `127.0.0.1` , so only programs on Kali can use the tunnel, not other machines on the network.

7.10 Useful variations

Command	Effect
`ssh -N -L 8888:127.0.0.1:3128 youruser@192.168.10.1`	Kali listens on port 8888 instead. The browser proxy must then use

Command	Effect
	`127.0.0.1:8888`.
`ssh -f -N -L 3128:127.0.0.1:3128 youruser@192.168.10.1`	`-f` runs the tunnel in the background.
`ssh -N -L 3128:127.0.0.1:3128 -p 2222 youruser@192.168.10.1`	`-p` is used when the SSH server listens on a port other than 22.
`ssh -N -L 3128:127.0.0.1:3128 -i ~/.ssh/id_ed25519 youruser@192.168.10.1`	`-i` selects a specific SSH key.
Adding `-v`	Prints debug output for troubleshooting.

Using SSH keys instead of an SSH password is better practice (`ssh-keygen` , then `ssh-copy-id youruser@192.168.10.1`).

7.11 Troubleshooting

Message	Meaning and fix
`bind [127.0.0.1]:3128: Address already in use`	Another program or an old tunnel uses local port 3128. Stop it or choose another local port.
`Connection refused` on `192.168.10.1:22`	The SSH server is not running on Ubuntu. Install and start `openssh-server`.
`channel 2: open failed: connect failed: Connection refused`	The tunnel works, but Squid is not listening on `127.0.0.1:3128`. Check `http_port` and restart Squid.
`administratively prohibited`	SSH forwarding is disabled. Check that `AllowTcpForwarding` is not set to `no` in `/etc/ssh/sshd_config`.

7.12 Other protection methods

- **TLS on the proxy (`https_port`):** Squid encrypts the client-to-proxy link itself. It needs a certificate (a self-signed one is acceptable in a lab, containing `IP:192.168.10.1`), the client must support HTTPS proxies (curl does; browsers usually need a PAC file or extra setup), and Squid must be built with TLS support.
- **VPN (for example WireGuard):** encrypts all traffic between the machines, not only the proxy, but takes more setup.
- **Stronger authentication methods** such as Kerberos or NTLM instead of Basic.

Property	SSH tunnel	`https_port`	VPN
Setup effort	Low	Medium	Higher
Needs a certificate	No	Yes	No (uses keys)
Squid sees real client IP	No	Yes	Yes (VPN IP)
Protects other traffic	No	No	Yes

For this lab, the SSH tunnel was the quickest method to implement.

8. Applying the Configuration

After modifying the Squid configuration, the following commands were used to validate and apply the settings.

Configuration validation

```
sudo squid -k parse
```

This command instructs Squid to parse its configuration file.

Reloading the service

```
sudo systemctl reload squid
```

This reloads the running Squid service so that it can apply the updated configuration.

Checking the service

```
sudo systemctl status squid
```

This command displays the current state of the Squid service.

9. Testing from Kali Linux

The proxy was tested from Kali Linux using `curl`.

The `-x` option specifies the proxy address, while `-I` requests the response headers. The `-U` option supplies the proxy username and password.

9.1 Testing the permitted website

The following command sends a request to Kali Linux's official website through Squid:

```
curl -I -x http://192.168.10.1:3128 https://www.kali.org/
```

The test returned a successful HTTPS tunnel establishment followed by a successful response from the destination website.

This demonstrated that Kali Linux could access the permitted website through the proxy.

9.2 Testing a blocked website

The following command tests access to YouTube:

```
curl -I -x http://192.168.10.1:3128 https://www.youtube.com/
```

With the blocking policy active, the request returned `403 Forbidden` with the Squid access-denied indicator.

This demonstrated that Squid could reject requests matching a configured blocked domain.

9.3 Testing authentication

With authentication enabled, three tests were made.

No credentials:

```
curl -I -x http://192.168.10.1:3128 https://www.kali.org/
```

Expected result: `407 Proxy Authentication Required`. The proxy asks the client to log in.

Wrong password:

```
curl -I -x http://192.168.10.1:3128 -U alice:wrongpass https://www.kali.org/
```

Expected result: `407 Proxy Authentication Required`. The credentials were rejected.

Valid credentials:

```
curl -I -x http://192.168.10.1:3128 -U alice:correctpass https://www.kali.org/
```

Expected result: the tunnel is established and the website responds normally.

Blocked site with valid credentials:

```
curl -I -x http://192.168.10.1:3128 -U alice:correctpass
https://www.youtube.com/
```

Expected result: `403 Forbidden`. A valid login does not override the blocklist, because the blocklist rule comes first.

These tests demonstrated that the proxy refuses anonymous and incorrectly authenticated clients, and that authentication works together with the domain-blocking policy.

10. Monitoring Proxy Activity

Squid's access log was used to observe client requests.

The following command displays new log entries in real time:

```
sudo tail -f /var/log/squid/access.log
```

The access log provides information about requests processed by Squid, including destination addresses, client activity, and request results.

With authentication enabled, the log also shows the **username** of each request (the username field shows `-` for unauthenticated requests). Typical result codes include:

- `TCP_DENIED/407`: the client did not provide valid credentials.
- `TCP_DENIED/403`: the request was denied by an access rule, such as the blocklist.
- `TCP_TUNNEL/200`: an HTTPS tunnel was established successfully.

The service log can also be viewed with:

```
sudo tail -f /var/log/squid/cache.log
```

These logs are useful for observing proxy activity and understanding how Squid handles client requests.

11. Results

The project demonstrated the following capabilities:

- Deployment of Squid on Ubuntu Server.
- Configuration of a forward proxy listening on TCP port 3128.
- Definition of a client subnet using an ACL.
- Creation of a website blocklist for Facebook and YouTube.

- Creation of an allowlist that permits `kali.org` while denying other destinations through Squid.
- Configuration of Basic proxy authentication using `basic_ncsa_auth` and an `htpasswd` password file.
- Use of the `proxy_auth REQUIRED` ACL and the `deny !authenticated` rule to require a valid login.
- Protection of the proxy login with an SSH local-forwarding tunnel, including use from a web browser.
- Validation and reloading of the Squid configuration.
- Testing proxy requests from Kali Linux using `curl`, with and without credentials.
- Monitoring client requests and authentication results through Squid's access log.

The tests confirmed that the client could successfully access the permitted website through Squid, that a request to a blocked website was denied under the applicable blocking policy, and that requests without valid credentials were refused with a `407` response.

12. Conclusion

This project provided practical experience implementing a Squid forward proxy in a VirtualBox home lab.

Ubuntu Server was used to host Squid, and Kali Linux was used to generate web requests through the proxy. Access-control lists were configured to demonstrate blocking selected websites, allowing only a specific domain, and requiring user authentication.

The project also covered configuration validation, service reloading, HTTP and HTTPS proxy requests, user authentication, and access-log monitoring.

The lab demonstrates how Squid can serve as a centralized point for controlling web access in a network. Authentication adds user identification on top of IP-based control. Because Basic authentication sends credentials without encryption, an SSH tunnel was used to protect the login between the client and the proxy; alternatives such as TLS (`https_port`) or a VPN can be used in larger deployments. The work establishes a foundation for further practical work in proxy administration and network security, such as per-user policies, safer port restrictions (`Safe_ports` and `SSL_ports`), TLS on the proxy port, and stronger authentication methods.